

Advanced Mode-Matching Optimization for Fabry-Perot Cavities

Codebase Architecture and Physical Modeling

System Documentation

July 17, 2026

Abstract

This document provides a comprehensive technical analysis of the Advanced FP Mode-Matching Studio codebase. The software automates the design and tolerance analysis of lens configurations required to optimally couple a Gaussian laser beam into a hemispherical Fabry-Perot cavity. By leveraging rigorous ABCD matrix propagation, overlap integrals, and Monte Carlo tolerance simulations, the suite offers a robust platform for preparing optical bench layouts. This is particularly critical when establishing a solid mode-matched foundation before engaging sensitive frequency stabilization protocols, such as Pound-Drever-Hall (PDH) locking to a reference cavity.

1 Introduction

The provided Python codebase is divided into three primary modules, operating harmoniously to provide a graphical user interface, physics calculations, and data serialization:

- **setup_data.py**: Defines the fundamental data structures and default optical parameters using Python dataclasses.
- **optimization.py**: The computational core, containing the physics engine (Gaussian beam propagation, cavity modes) and the numerical optimization algorithms.
- **gui.py**: A **tkinter**-based graphical front-end that handles user inputs, renders dynamic beam profile visualizations using **matplotlib**, and manages XML state persistence.

2 Physical Modeling (**optimization.py**)

The physics engine is built upon the paraxial approximation of Gaussian beams.

2.1 Gaussian Beam Propagation and ABCD Matrices

The beam is tracked along the optical axis (z) using the complex beam parameter q :

$$\frac{1}{q(z)} = \frac{1}{R(z)} - i \frac{\lambda}{\pi w(z)^2} \quad (1)$$

where $R(z)$ is the radius of curvature, $w(z)$ is the beam radius (waist), and λ is the wavelength. The function `get_beam_radius(q, wavelength)` parses this complex parameter to extract w and R .

When propagating through free space (distance d) or a thin lens (focal length f), the complex parameter transforms via ABCD matrices:

$$q_{out} = \frac{Aq_{in} + B}{Cq_{in} + D} \quad (2)$$

In the code, free space propagation is implemented simply as $q_{out} = q_{in} + d$ (since $A = 1, B = d, C = 0, D = 1$), and a thin lens is modeled as $q_{out} = q_{in}/(-\frac{1}{f}q_{in} + 1)$.

2.2 Cavity Eigenmodes

To determine the target beam shape, the code calculates the stable eigenmode of the hemispherical Fabry-Perot cavity using `calculate_cavity_modes`. For a cavity of length L , flat input mirror ($R_1 = \infty \implies C_1 = 0$), and curved output mirror (R_2), the stability parameters are $g_1 = 1$ and $g_2 = 1 - L/R_2$. The beam waists at the mirrors are computed as:

$$w_1^2 = \frac{L\lambda}{\pi} \sqrt{\frac{g_2}{g_1(1 - g_1g_2)}}, \quad w_2^2 = \frac{L\lambda}{\pi} \sqrt{\frac{g_1}{g_2(1 - g_1g_2)}} \quad (3)$$

2.3 Overlap Integral and Coupling Efficiency

Mode matching quality is quantified by the coupling efficiency η , evaluated by `calculate_coupling_efficiency`. It represents the spatial overlap between the incident beam (w_1, R_1) and the cavity eigenmode (w_2, R_2):

$$\eta = \frac{4w_1^2w_2^2}{(w_1^2 + w_2^2)^2 + \left(\frac{\pi w_1^2w_2^2}{\lambda}\right)^2 \left(\frac{1}{R_1} - \frac{1}{R_2}\right)^2} \quad (4)$$

The optimization routine actively seeks to minimize $(1 - \eta)$.

2.4 Vacuum Viewport Considerations

When dealing with vacuum-housed cavities, a tilted glass viewport introduces an optical path length shift. The function `get_window_shift(t, theta_deg, n)` calculates this longitudinal shift Δz :

$$\Delta z = t \left(1 - \frac{\cos \theta}{\sqrt{n^2 - \sin^2 \theta}} \right) \quad (5)$$

where t is thickness, θ is the tilt angle (to prevent back-reflections), and n is the refractive index.

3 Computational Optimization Routine

The optimization leverages a multithreaded approach combining grid search and gradient descent.

3.1 Grid Search & L-BFGS-B Minimization

`run_multithreaded_optimization` builds a combinatorial pool of possible lens focal lengths from a standard catalog (e.g., Thorlabs lenses). For each combination, it spawns a task via `ThreadPoolExecutor`.

Within `optimize_single_combination`, the `scipy.optimize.minimize` function utilizes the L-BFGS-B algorithm (suitable for bounded variables) to adjust the distances between the lenses and the cavity to maximize the coupling efficiency η .

3.2 Monte Carlo Tolerance Analysis

Because theoretical models often fail due to real-world mechanical and optical imperfections, `monte_carlo_tolerances` performs a robustness check. It samples 500 permutations of the system using uniformly distributed random errors for:

- Wavelength ($\pm\Delta\lambda$)
- Cavity length and mirror ROC ($\pm\Delta L$, $\pm\Delta R_2$)
- Lens positioning and focal lengths ($\pm\Delta d$, $\pm\Delta f$)
- Viewport thickness and angle ($\pm\Delta t$, $\pm\Delta\theta$)

This provides a "Worst-Case Efficiency" metric, ensuring the chosen lens configuration is insensitive to typical machining or alignment tolerances.

4 Software Architecture and State Management

4.1 Dataclass Configuration (`setup_data.py`)

The `OpticalSetup` class strictly defines the optical bench. Utilizing Python's `@dataclass`, it neatly encapsulates both the nominal values (e.g., `wavelength`, `cavity_length`) and their respective tolerance bounds (e.g., `tol_wl`, `tol_cl`).

4.2 GUI and XML Serialization (`gui.py`)

The Tkinter front-end operates asynchronously to prevent GUI lockups during heavy optimization.

- **Visualization:** `matplotlib.backends.backend_tkagg` embeds dynamically generated plots of the beam envelope $w(z)$ (`draw_sketch`) and a comparative plot of the top combinations (`draw_comparison`).
- **Persistence:** The methods `save_settings_to_xml` and `load_settings_from_xml` utilize the `xml.etree.ElementTree` module to serialize the UI state into `last_settings.xml`, ensuring continuity between sessions.

5 Conclusion

This codebase represents a specialized, end-to-end engineering tool. By marrying theoretical complex beam propagation with high-performance numerical optimization and tolerance modeling, it eliminates the guesswork involved in mode-matching high-finesse Fabry-Perot cavities.